



GPU FinOps & Cost Optimization

AICB-P2T2 · Ngày 25 · Chương 5: Vận Hành

Giảng viên

VinUniversity · Phase 2 · Track 2 · Tuần 5

HÃY SUY NGHĨ...

*“Bạn đang tiêu bao nhiêu cho GPU mỗi ngày? Và bao nhiêu % là lãng phí? **Case study:** 4x A100 idle overnight (12h) = \$144 wasted/day = \$52,560/year. Hôm nay chúng ta học cách cắt giảm 40–60% chi phí GPU – và close Chương 5 với Quiz + Milestone 2.”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. GPU Cloud Cost Anatomy
2. Spot & Preemptible Instances
3. Right-Sizing & Utilization
4. Inference Cost Optimization
5. Cost Allocation & Chargeback
6. Sustainable AI: Carbon & Energy
7. Tổng Kết Chương 5
8. **Quiz + Milestone 2**

Sau buổi học này, bạn sẽ:

1. Phân tích chi phí GPU cloud và phát hiện lãng phí
2. Áp dụng spot/preemptible instances với checkpoint strategy
3. Tối ưu inference cost bằng batching, caching, model cascading
4. Thiết kế cost allocation & FinOps review process cho team

Cost anatomy → Spot strategy → Right-sizing → Inference optimization →
FinOps governance → Quiz + Milestone 2

Cost optimization plan + projected savings + Quiz + Milestone 2 platform demo

- Cost baseline report: GPU utilization, idle time, wasted spend hiện tại
- Optimization plan: spot strategy, right-sizing, batching – projected savings
- **Quiz Chương 5:** 15 câu hỏi CI/CD, LLMOps, Monitoring, Governance, FinOps
- **Milestone 2:** Demo operations platform end-to-end

GPU Cloud Cost Anatomy



Hiểu bill GPU không chỉ là giá card theo giờ; hidden cost, idle cost, và over-provisioning mới là nơi thất thoát ngân sách lớn nhất.



- **Hidden costs:** data transfer egress (\$0.09/GB AWS), NAT gateway (\$0.045/GB), Secrets Manager (\$0.40/secret/mo)
- **Wasted spend:** idle GPUs (training done, instance running), over-provisioned instances, unused reserved capacity
- **FinOps maturity:** Inform → Optimize → Operate – hầu hết AI teams chỉ ở Inform level

1. GPU idle overnight – training xong nhưng instance vẫn chạy
2. Over-provisioned: dùng H100 cho inference 8B model (A10G đủ rồi)
3. Unused reserved capacity mua 1 năm nhưng workload thay đổi
4. Development environments chạy 24/7 (chỉ cần giờ hành chính)

- 4x A100 idle 12h/day
- Chi phí: $4 \times \$3.0/\text{hr} \times 12\text{h} = \$144/\text{ngày}$
- **\$52,560/năm** – chỉ riêng idle time!
- Fix: auto-shutdown schedule → tiết kiệm ngay

Rule of thumb: GPU utilization $< 30\%$ = cần right-size ngay

Spot & Preemptible Instances

Tiết kiệm lớn nhất thường đến từ chấp nhận gián đoạn có kiểm soát, kết hợp mixed fleet và checkpoint strategy đúng cách.

- Discount 60–70% so với on-demand
- 2-min termination notice
- Spot Fleet: request từ nhiều AZ/instance types
- Giảm interruption rate từ 15% → 3%

- Discount tới 80%
- Terminate sau 24h (Preemptible) hoặc flexible (Spot)
- Phù hợp jobs <20h
- Tự động reschedule trên GKE

Mixed fleet strategy: 20% on-demand (baseline) + 80% spot (burst) – balance cost vs reliability **SkyPilot:**

Multi-cloud spot abstraction layer – tự động tìm cheapest spot across AWS/GCP/Azure. `sky launch task.yaml` chọn provider tối ưu.

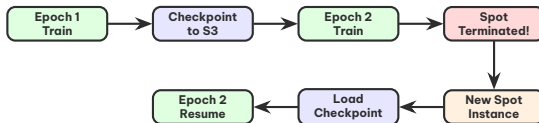
Metric	Đo gì?	Hợp nhất khi nào?	Nếu thấp thì làm gì?
GPU Utilization	GPU có bận hay không	Audit idle, phát hiện máy bỏ không	Tắt máy idle, gom workload, tăng batch-ing
MFU	FLOPs thực tế / FLOPs đỉnh	Training, prefill, workload compute-bound	Tối ưu kernel, batch size, data loading
MBU	Băng thông nhớ thực tế / đỉnh	Decode, serving, workload memory-bound	KV cache, quantization, disaggregation, prefix reuse

Cảnh báo: GPU utilization 95% chưa chắc tốt; có thể GPU chỉ đang bận chờ HBM thay vì tạo ra nhiều token hữu ích.

- Dễ hiểu cho budgeting và chargeback
- Hợp để soi idle time, spot mix, right-sizing
- Nhưng chưa phản ánh tốc độ sinh token
- Hai cụm cùng \$10/h có thể khác xa hiệu quả

- Công thức: $\$/\text{token} = \$/\text{giờ} / (\text{tok/s} \times 3600)$
- So sánh công bằng giữa model, GPU, engine
- Tăng tok/s $\rightarrow$ cost/token giảm ngay cả khi cost/giờ giữ nguyên
- Phù hợp hơn cho bài toán tối ưu inference LLM

Best practice: dashboard phải hiển thị cả **\$/giờ**, **\$/1M token**, TTFT, ITL, cache hit rate để tránh tối ưu sai mục tiêu.



- Save model state mỗi epoch (hoặc mỗi 30 phút cho long epochs)
- Checkpoint lưu lên S3/GCS – resume từ bất kỳ instance nào
- PyTorch Lightning `ModelCheckpoint` callback tự động hoá
- Best practice: test resume flow **trước** khi chạy long training

Right-Sizing & Utilization

Tối ưu bắt đầu bằng đo đúng. Sau đó mới nói đến MIG, multi-model serving, batching, và mọi quyết định cấp phát GPU.

Workload	Typical Util	Target	Action nếu thấp
Inference (single model)	20–40%	>60%	Multi-model serving, MIG
Inference (batched)	50–70%	>75%	Tune batch size, queue
Fine-tuning	60–80%	>80%	Larger batch, gradient accum
Pre-training	80–95%	>90%	Optimise data loading

Monitoring: `nvidia-smi dmon -d 5` hoặc DCGM Exporter → Prometheus → Grafana dashboard

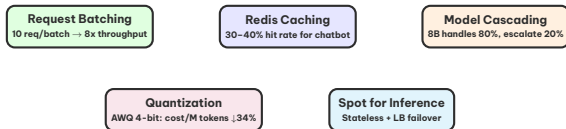
- vLLM serve Llama-3-8B + Mistral-7B trên A10G (24GB)
- Dynamic loading: swap models theo request
- Utilization tăng từ 25% → 65%

- A100 80GB → 7 isolated instances (3g.20gb)
- 7 models chạy song song, isolation đảm bảo
- K8s: `nvidia.com/gpu.shared: true`
- Perfect cho inference farm

Vertical Pod Autoscaler (VPA): recommend CPU/memory limits dựa trên actual usage – avoid over-provisioning resources.

Inference Cost Optimization

Sau khi đo đúng, ta tối ưu toàn bộ stack inference: batching, cache, disaggregation, routing, quantization, và economics theo cost/token.



Phần mở rộng ngay sau đây: disaggregated serving,

chunked prefilling, speculative decoding, KV cache reuse, NIMs, và cost/token.

- Group 10 requests mỗi batch
- Throughput tăng 8x, cost/request giảm 85%
- vLLM continuous batching tự động
- Tune `max_num_seqs` theo latency SLO

- Redis cache cho identical prompts
- Hit rate 30–40% điển hình cho chatbot
- Semantic cache: embed prompt → similarity search
- TTL: 1h cho dynamic, 24h cho static prompts

- Small model (Llama-3-8B) xử lý 80% requests
- Escalate 20% complex queries → large model
- Router: classify difficulty bằng fast classifier
- Cost reduction: 60–70%

Mode	Tok/s	\$/M tok
FP16 (A10G)	1200	\$0.83
AWQ 4-bit	1800	\$0.55
Savings		34%

Spot cho inference: stateless inference servers phù hợp spot instances nếu có load balancer automatic failover.

- **Prefill** xử lý prompt dài → thiên về compute
- **Decode** sinh token tuần tự → thiên về memory bandwidth
- Dồn cả hai vào cùng GPU thường làm nghẽn tài nguyên
- Prompt càng dài, lệch pha càng rõ

- Tách cụm GPU **prefill** và **decode** riêng
- Scale độc lập theo **TTFT** và **ITL**
- Hợp với long-context, agentic, RAG, reasoning
- Hỗ trợ stack như TensorRT-LLM, vLLM, SGLang

Khi nên cân nhắc: khi prompt dài, queue biến động mạnh, hoặc decode đang bóp nghẹt throughput của cả cụm.

- Chia prompt dài thành nhiều **chunk** nhỏ
- Ưu tiên decode requests trước
- Phần token budget còn lại mới nạp thêm prefill
- Tránh để prompt dài chặn hàng đợi decode

- Tốt cho traffic hỗn hợp: prompt dài + chat liên tục
- Giảm tail latency và tăng độ mượt khi sinh token
- Chunk quá nhỏ → TTFT xấu đi
- Chunk quá lớn → decode vẫn bị chen ngang

Rule of thumb: bắt đầu tune theo `max_num_batched_tokens`, đo đồng thời TTFT, ITL, queue depth.

- Một **draft model** dự đoán trước vài token
- Model chính xác minh cả cụm token trong một lượt
- Nếu khớp nhiều → ITL giảm rõ rệt
- Hợp với workload decode đang memory-bound

- Hiệu quả cao hơn ở medium/low QPS, latency-sensitive
- Cần thêm draft model hoặc MTP/EAGLE backend
- Không phải lúc nào cũng tăng throughput giờ cao điểm
- Theo dõi **acceptance rate** và tok/s thực tế

Đừng bật mù quáng: nếu acceptance thấp, bạn chỉ đang trả thêm chi phí xác minh mà không thu được tốc độ.

- Lưu key/value của các token đã xử lý
- Decode token tiếp theo không phải tính lại toàn prompt
- Đổ lại: ăn VRAM, dễ tạo áp lực admission/eviction
- Có thể offload sang host nếu engine hỗ trợ

- Tái sử dụng phần **đầu prompt giống nhau** giữa nhiều request
- Rất hợp với system prompt dài, tài liệu lặp lại, RAG cố định
- Với NVIDIA NIM, bật
NIM_ENABLE_KV_CACHE_REUSE=1
- Cache hit tốt → TTFT request thứ 2 giảm mạnh

Metric bắt buộc: cache hit rate, KV memory usage, eviction rate, TTFT trước/sau khi bật cache.

- Microservice inference dựng sẵn cho model
- Runtime production-grade, update bảo mật liên tục
- Chạy trên cloud hoặc data center có GPU NVIDIA
- Giảm công tích hợp hạ tầng từ đầu

- Team cần **time-to-production** nhanh
- Muốn chuẩn hoá observability, auth, lifecycle
- Có thể bật KV cache reuse trực tiếp bằng biến môi trường
- Ít phù hợp nếu bạn cần tối ưu cực sâu từng kernel

Cách nhìn thực dụng: NIM giảm chi phí vận hành đội platform; Dynamo/vLLM thường tối ưu sâu hơn về cost/token ở quy mô lớn.

- Release chính thức: **24/04/2026**
- **V4-Pro**: 1.6T total / 49B active params
- **V4-Flash**: 284B total / 13B active params
- Cả hai hỗ trợ **1M context**

- Flash: \$0.14 miss / \$0.0028 hit / \$0.28 output
- Pro: \$0.435 miss / \$0.003625 hit / \$0.87 output
- Cache-hit của Flash rẻ hơn miss **50x**
- Cache-hit của Pro rẻ hơn miss **120x**

Kết luận ngắn: DeepSeek rẻ không chỉ vì “GPU rẻ” mà vì giảm cả **activated compute**, **KV memory**, và **chi phí input lặp lại**.

- **CSA:** nén KV rồi sparse-select top-k
- **HCA:** nén KV mạnh hơn nhưng vẫn dense attention
- Kiến trúc hybrid giúp 1M context khả thi
- Hợp với long-context, agentic, reasoning

- **V4-Pro @1M:** 27% single-token FLOPs, 10% KV cache
- **V4-Flash @1M:** 10% single-token FLOPs, 7% KV cache
- Giảm FLOPs → giảm cost/token
- Giảm KV → giảm áp lực VRAM và bandwidth

- Không kích hoạt toàn bộ 1.6T hay 284B params mỗi token
- Thực tế chỉ dùng **49B** hoặc **13B** active params
- DeepSeekMoE giữ chất lượng nhưng giảm compute thực chạy
- Đây là lý do cost/token thấp hơn dense model cùng tầm

- MoE expert weights dùng **FP4**
- KV cache dùng hybrid **BF16 + FP8**
- Indexer attention path dùng **FP4**
- Ít memory traffic hơn → MBU tốt hơn

- Official paper nhấn mạnh **CSA + HCA**
- **MLA không phải** điểm nhấn chính của V4 paper
- Có heterogeneous KV cache + on-disk shared-prefix reuse
- Muon + mHC giúp train ổn định và hội tụ nhanh hơn

- **Redis cache** hữu ích ở app layer nếu self-host
- **Huawei/Ascend** có thể giúp hạ TCO self-host
- Nhưng đây **không phải** luận điểm core trong docs pricing
- Dạy học viên phải tách model economics và infra economics

Thông điệp dạy học: DeepSeek rẻ vì nhiều lớp tối ưu chồng lên nhau; đừng quy toàn bộ vào một buzzword duy nhất.

- team=ml-platform
- project=rag-service
- env=production
- cost-center=engineering
- Enforce tags bằng SCP/OPA policies

- ResourceQuota per namespace
- Team budget: max 4 GPUs, 100GB storage
- **Kubecost:** per-pod cost breakdown
- “RAG service \$45/day, Embedding \$12/day”

AWS Cost Explorer + Budgets: alert khi spending vượt \$1,000/day → immediate investigation. **Monthly FinOps review:** share dashboard → ownership drives optimization.

- Chạy training khi grid sạch nhất
- us-west-2 (Oregon hydro) 10x greener than us-east-1
- CodeCarbon library: track CO₂ per experiment
- Schedule heavy jobs off-peak hours

- Distilled models: Phi-3-mini vs GPT-4 – 100x smaller, 70% accuracy
- Chain-of-thought tăng cost 3-5x – chỉ dùng khi cần
- Green AI metric: CO₂ grams per 1000 inferences
- Track & report cùng performance metrics

Takeaway: Sustainable AI không chỉ “nice-to-have” – region selection + model selection = cost savings + carbon reduction cùng lúc.

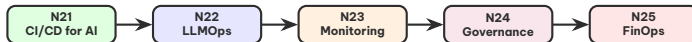
1. **Bước 1 – Audit:** `nvidia-smi dmon` → đo GPU utilization hiện tại, phát hiện idle
2. **Bước 2 – Batching:** Enable request batching, benchmark throughput trước/sau
3. **Bước 3 – Kubecost:** Cài Kubecost, tag resources, xem per-service cost breakdown
4. **Bước 4 – Spot:** Viết checkpoint/resume logic cho training job trên spot instance
5. **Bước 5 – Report:** Tạo cost optimization report: baseline vs optimized, projected savings

Mục tiêu: GPU FinOps Optimization Workshop

Deliverable: Cost optimization report (baseline vs optimized) + Milestone 2 platform demo

Thời gian: 2.5h

Chương 5: Operations Layer Complete



- **Key insight:** operations cost thường = infrastructure cost sau 6 tháng production
- **FinOps quick wins:** schedule idle shutdown + quantize models + implement caching = 40–60% reduction
- **Invest early:** CI/CD + monitoring + governance + FinOps – trả nợ sớm, không phải trả lãi

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 GPU cost anatomy: Compute 60% + hidden costs (egress, NAT, secrets) – audit trước khi optimize.
- 2 Spot instances tiết kiệm 60–70% – checkpoint mỗi 30 phút, mixed fleet 20/80 on-demand/spot.
- 3 Inference optimization: batching + KV cache + disaggregation + speculation + quantization → giảm mạnh cost/token nếu đo đúng TTFT, ITL, MBU.

Chương 6: Tổng Hợp – MCP/A2A Infrastructure

“Agent gọi agent – MCP server hosting, A2A protocol, agentic routing cho multi-agent systems”

- Hoàn thành Lab 25 + Milestone 2 demo
- Đọc trước: Anthropic MCP specification
- Đọc trước: Google A2A protocol overview

Hỏi & Đáp

Câu hỏi nào về GPU FinOps, spot instances, inference optimization, hay cost allocation?



Cảm ơn!

AICB-P2T2 · Ngày 25

GPU FinOps & Cost Optimization

lms.vinuni.edu.vn · Slide & template trên LMS